

Actividad 1 — Puntos de dolor de la arquitectura actual

Reto 2 · Patrones de diseño arquitectónico Equipo: Santiago HM, Simón BU, Sebastián QJ

Qué código se analizó

El AS-IS de este reto es el código que entregamos en el Reto 1, no el original de la hacienda. Vive en:

```
03-src/redisenado/HaciendaNEW/
├─ Bib_Hacienda/      biblioteca de dominio
└─ p_mvcHacienda/     aplicación MVC
```

Todas las referencias de archivo y línea de este documento apuntan ahí. Se verificaron el 5 de septiembre de 2026 con los comandos de la sección 4.

Una advertencia para quien revise el repositorio: la carpeta `05-reto2-patrones/04-src/baseline-input-2026-08-31/` tiene nombre de baseline pero no lo es. Contiene ya aplicados Factory Method, Builder y Observer, más el esqueleto de `HistoriaClinica`. Si buscas ahí los switches de P-01 o las suscripciones de P-03 no los vas a encontrar, porque ya están resueltos. El AS-IS real es `03-src/redisenado/`.

1. Puntos de dolor

Siete encontrados. El enunciado pide cinco, así que en el documento final hay margen para descartar dos.

ID	Dónde	Qué lo hace caro	Cuánto cuesta hoy	Prioridad
P-01	Bib_Hacienda/Clases/Potrero.cs:40-140 (anadir_res) y :202 (agregar) p_mvcHacienda/Servicios/PersistenciaService.cs:512-526 y :571-574	Para dar de alta una res, <code>anadir_res</code> primero traduce el enum del potrero a un string (<code>tipo_vaca</code> , línea 53) y después hace un segundo switch sobre ese texto para ejecutar el <code>new</code> (línea 90). Los dos switches no están atados: si añades un caso al primero y olvidas el segundo, <code>res</code> queda en null, el método sigue y el fallo aparece después dentro de un publisher con el mensaje "Error inesperado en el metodo Informar_Peso_Min", que no dice nada del origen. La misma decisión se repite dos veces en <code>PersistenciaService</code> , y ahí el switch de recarga termina en <code>_ => new Ternero(...)</code> : cualquier tipo desconocido guardado en disco vuelve convertido en ternero, en silencio. Aparte, <code>agregar</code> (línea 202) es una segunda puerta al mismo potrero, con las reglas de edad reescritas y una validación de duplicados que <code>anadir_res</code> no tiene.	Escenario: añadir una cuarta categoría de res. 3 clases y 4 archivos: <code>Potrero</code> , <code>PersistenciaService</code> , la clase nueva y <code>Views/Potrero/Create.cshtml</code> . Dentro de ellos, 5 bloques de decisión que hay que sincronizar a mano.	Alta
P-02	Eventos/PublisherPesoMin.cs:25-27 Eventos/PublisherPesoVenta.cs:27-29 Eventos/PublisherVacunacionCompletada.cs:30-40 Servicios/ResService.cs:97-99 Views/Res/Index.cshtml:69-71, Views/Venta/Index.cshtml:107-117	Para saber el peso mínimo de una res, seis archivos preguntan <code>res is Ternero / is Cebon / is Novillo</code> y eligen la constante a mano. Lo llamativo es que <code>Res</code> ya resolvió bien este problema para el cupo de vacunas: <code>MaxVacunasBacterianas</code> es abstracta y cada subtipo la responde. Peso mínimo, peso de venta y esquema completo no siguieron ese camino.	Mismo escenario que P-01. 6 clases y 8 archivos: los 3 publishers, <code>ReglaRes</code> , <code>ReglaVacuna</code> , <code>ResService</code> y 2 vistas. Sumado a P-01: 9 clases y 12 archivos para un solo tipo nuevo.	Alta

ID	Dónde	Qué lo hace caro	Cuánto cuesta hoy	Prioridad
		Ninguna de las cadenas tiene rama final, así que si no coincide ningún tipo la variable queda en 0 y el aviso no se dispara: <code>PublisherPesoMin</code> daría por bien alimentada a una res desnutrida sin error visible.		
P-03	<code>Clases/Potrero.cs:112, 118, 124, 130</code> <code>Clases/Hacienda.cs:225, 231, 294</code>	Las suscripciones a eventos (<code>+=</code>) están escritas dentro del cuerpo de los métodos, sobre publishers que son campos de instancia, y nunca se sueltan. <code>Hacienda</code> está registrada como <code>AddSingleton</code> , así que vive lo que dura el proceso: en la llamada número 200 a <code>alimentar_res</code> se ejecutan 200 handlers, 199 de ellos escribiendo en variables locales de invocaciones ya terminadas. La lista de suscriptores solo crece. Y para saber quién escucha un evento no basta con leer la declaración de la clase, hay que leer el cuerpo de cada método.	Escenario: añadir un aviso nuevo, por ejemplo "res lista para traslado". 8 clases y 8 archivos solo para entender el mecanismo: <code>Hacienda</code> , <code>Potrero</code> y los 6 publishers. 7 puntos de suscripción repartidos en 3 métodos.	Alta
P-04	<code>Clases/FabricadorVacunas.cs:60 y :98</code> (los dos <code>CrearLote</code>) <code>Interfaces/ICreacionVacuna.cs:13-16</code> <code>Servicios/VacunaService.cs:26-36</code>	<code>CrearLote</code> está escrito dos veces, idéntico salvo la línea del <code>new</code> y el texto del mensaje. La copia bacteriana arrastra un error que la viva no tiene: en la línea 86 falta el <code>\$</code> de la interpolación, así que el resumen del lote imprime literalmente <code>- Nombre: {nombre}</code> . Nadie lo ha notado porque hay que leer las dos versiones en paralelo para verlo. Las cuatro firmas están además publicadas en <code>ICreacionVacuna</code> , de modo que un tipo nuevo de vacuna obliga a cambiar el contrato y con él todo lo que lo implementa. <code>VacunaService</code> decide qué clase concreta crear mirando cuál de los dos parámetros opcionales vino con valor, que es un tercer sitio donde se toma la misma decisión.	Escenario: incorporar un tercer tipo de vacuna. 7 clases y 11 archivos: la clase nueva, <code>FabricadorVacunas</code> , <code>ICreacionVacuna</code> , <code>Hacienda</code> , <code>VacunaService</code> , <code>VacunaController</code> , <code>PersistenciaService</code> y 4 vistas.	Alta
P-05	<code>Clases/Res.cs:104-105</code> <code>Ternero.cs:27,30</code> , <code>Novillo.cs</code> , <code>Cebon.cs</code> <code>enum/TipoVacuna.cs</code>	<code>Res</code> declara una propiedad abstracta por cada tipo de vacuna que existe, respondida por las tres subclases: 6 propiedades para cubrir 2x3 combinaciones, y cada tipo nuevo de vacuna añade 4 más. El enum <code>TipoVacuna</code> repite la lista de subclases de <code>Vacuna</code> , así que el mismo dato vive en dos sitios que nadie obliga a mantener sincronizados. Si se crea la clase y se olvida el enum, <code>Res.aplicar_vacuna</code>	Mismo escenario que P-04. 6 clases y 6 archivos: <code>Res</code> , las 3 subclases, <code>TipoVacuna</code> , <code>ReglaVacuna</code> . Sumado a P-04: 17 archivos para un tipo nuevo de vacuna.	Media

ID	Dónde	Qué lo hace caro	Cuánto cuesta hoy	Prioridad
		cae en el default de su switch y devuelve un mensaje que no dice cuál era el límite.		
P-06	Clases/Autenticacion.cs (150 líneas) Interfaces/IAutenticacion.cs Infraestructure/InterceptorAutenticacion.cs (69 líneas) Servicios/UsuarioService.cs	Hay dos sistemas de usuarios en paralelo y el que tiene las reglas de permisos no está conectado. Autenticacion es la única clase que sabe qué puede hacer cada rol, y nadie la instancia: new Autenticacion no aparece en ningún archivo del proyecto. InterceptorAutenticacion tampoco está registrado, solo se define a sí mismo. Quien autentica de verdad es UsuarioService , que guarda los usuarios en un static List<Usuario> y no comprueba ningún permiso. Son 219 líneas que parecen ser el control de acceso del sistema y no se ejecutan nunca: quien quiera cambiar permisos va a editar el archivo equivocado y no va a notar nada. Aparte, AutorizarOperacion decide el rol comparando el nombre de usuario contra los literales "admin", "empleado" y "visitante", así que un usuario creado desde la aplicación no puede tener permisos, y comunica el resultado lanzando excepción también cuando autoriza.	Escenario: "el veterinario puede aplicar vacunas pero no vender". 5 clases y 6 archivos. Antes de tocar nada hay que leer 219 líneas muertas para descubrir que no corren.	Media
P-07	Clases/Hacienda.cs:182-202 (vender_res) y el vender<T> genérico Clases/InventarioCarnes.cs, InventarioLacteos.cs, InventarioPiel.es (64 líneas cada una) Servicios/ResService.cs:79	La venta de productos derivados está construida y desconectada. Existen los tres inventarios y un vender<T> genérico, y ninguno se instancia ni se llama desde la aplicación: new InventarioCarnes no aparece en ningún sitio. El único camino de venta vivo es vender_res , el viejo. Las tres clases de inventario son el mismo archivo copiado tres veces, 192 líneas para tres comportamientos idénticos, y ya divergieron en detalles. Los dos formatos de venta conviven dentro de Venta , así que ValidadorVenta tiene que preguntar cuál de los dos trae cada registro antes de saber si es válido.	Escenario: vender un lácteo desde la aplicación. 8 clases y 8 archivos solo para averiguar cuál de los dos caminos está vivo. Más 1 servicio, 1 controlador y 2 vistas para conectarlo.	Media

2. Criterio de priorización

La prioridad no es una impresión. Se calcula así:

archivos que hay que abrir x cercanía al Anexo B

Cercanía vale 2 si una solicitud de cambio del Anexo B toca ese punto de forma directa, 1 si lo toca de lado, 0 si no lo toca.

ID	Archivos	Anexo B	Cercanía	Puntaje	Prioridad
P-01	12 (con P-02)	SC-2 pide colgar un chip de cada res, o sea crear y recargar reses	2	24	Alta
P-02	8	SC-2, mismo motivo	2	16	Alta
P-04	11	SC-3 amplía el registro clínico, y las vacunas son la mitad de ese registro	2	22	Alta
P-03	8	SC-2, un chip de geolocalización genera avisos nuevos	2	16	Alta
P-05	6	SC-3, de lado	1	6	Media
P-07	8	SC-1, ya implementada en el Reto 1	1	8	Media
P-06	6	Ninguna solicitud toca permisos	0	0	Media

P-06 queda en media pese a puntuar cero porque 219 líneas muertas que aparentan ser el control de acceso son una trampa para cualquiera que entre al proyecto. Es un costo de lectura, no de cambio, y el criterio numérico no lo captura. Lo dejamos anotado como excepción declarada en vez de forzar la fórmula.

3. Punto que no se interviene

El enunciado exige mínimo uno. Proponemos N-02 como el que va al documento, y dejamos N-01 y N-03 como alternativas por si el equipo prefiere otro.

ID	Dónde	Qué lo hace caro	Cuánto cuesta hoy
N-01	<code>Servicios/PersistenciaService.cs</code> (654 líneas, 5 interfaces)	Un solo archivo lee y escribe los seis archivos de <code>Datos/</code> , cada uno con su formato separado por <code> </code> , sin cabecera ni versión. Los métodos de carga llevan <code>try/catch</code> línea por línea, así que una línea corrupta se salta en silencio y el sistema arranca con menos datos de los que hay en disco.	1 clase, 654 líneas, 6 formatos. Intervenirlo costaría unas 12 clases nuevas.
N-02	32 sitios en 14 clases (<code>Hacienda</code> 5, <code>Potrero</code> 4, <code>FabricadorVacunas</code> 4, los 3 inventarios 3 c/u, <code>Res</code> 2, <code>Autenticacion</code> 2, los 6 publishers 1 c/u)	Cada método envuelve la excepción del método al que llamó con <code>catch (Exception er) { throw new Exception("Error inesperado en X: " + er.Message); }</code> . Un fallo al añadir una res llega a la vista como "Error inesperado en el metodo anadir_res_potrero: Error inesperado en el metodo anadir_res: ...", con el mensaje real al fondo y la traza original perdida.	14 clases, 32 sitios.
N-03	7 vistas: <code>Res/Index.cshtml:69-71</code> , <code>Venta/Index.cshtml:107-117</code> , <code>Potrero/Create.cshtml:46-48</code> , <code>Vacuna/Index.cshtml:76-83</code> , <code>Vacuna/Aplicar.cshtml:67</code> , <code>Vacuna/Create.cshtml:39-46</code> , <code>Res/DetalleVacunas.cshtml:42-49</code>	La etiqueta y el color de cada tipo de res se deciden dentro de la vista, con cadenas de <code>@if (item.Res is Ternero)</code> . Cada tipo nuevo obliga a abrir siete archivos de presentación y acordarse de los siete, sin que nada avise si se olvida uno.	7 vistas de las 22 del proyecto.

Por qué no se interviene N-02

Estos mensajes son la salida observable del sistema: llegan tal cual a las vistas del usuario. La regla 2 del enunciado descuenta 0.5 sobre la nota final por cada cambio no autorizado en el comportamiento observable, así que limpiarlos es exactamente lo que no podemos hacer sin permiso.

El remedio correcto son excepciones de dominio tipadas. Eso toca 14 clases y 32 sitios, reescribe texto congelado y no resuelve ninguna solicitud del Anexo B. Mucho trabajo, riesgo de penalización directa y cero beneficio para el negocio. Lo declaramos como deuda técnica conocida y pedimos autorización, no lo arreglamos este trimestre.

Los otros dos, en corto

N-01 queda fuera porque el alcance excluye "base de datos real" y esta capa se reemplaza entera el día que exista una. Pagariamos doce clases de indirección sobre algo que se va a tirar. Además, la parte de este archivo que sí duele, el switch con su `_ => new Ternero(...)`, ya está contada en P-01 y se arregla desde ahí sin tocar las otras 600 líneas.

N-03 queda fuera porque el alcance excluye "interfaz gráfica" y el correo acota el encargo a cómo colaboran los objetos dentro del back. Meter un patrón en la capa de presentación arriesga el tope de 2.5 del criterio 3 por cambio de estilo. El costo real es bajo: son siete etiquetas de texto, y cuando se olvida una el fallo es cosmético y salta a la vista.

4. Cómo se midieron los costos

Los conteos salen de estos comandos, ejecutados sobre `03-src/redisenado/HaciendaNEW/` excluyendo `obj/` y `bin/`. Si en la sustentación los preguntan al azar, se vuelven a correr.

```
# P-01 - sitios que instancian un subtipo de Res
grep -rn "new Ternero\|new Novillo\|new Cebon" --include="*.cs" .
```

```
# P-02 – sitios que discriminan por subtipo
grep -rn "is Ternero\\|is Novillo\\|is Cebon" --include="*.cs" --include="*.cshhtml" .

# P-03 – puntos de suscripción a eventos
grep -rn "evt_.* +=" --include="*.cs" .

# P-06 – quién usa realmente la autenticación del dominio
grep -rn "new Autenticacion\\|InterceptorAutenticacion" --include="*.cs" .

# P-07 – quién instancia los inventarios y quién llama a vender
grep -rn "new Inventario\\|vender_res\\|.vender(" --include="*.cs" .

# N-02 – excepciones envueltas, por clase
grep -rc "Error inesperado en" --include="*.cs" .
```

Resultados verificados el 5 de septiembre de 2026:

Métrica	Valor
Archivos a abrir para añadir un tipo de res	12 (P-01 + P-02)
Archivos a abrir para añadir un tipo de vacuna	17 (P-04 + P-05)
Puntos de suscripción a eventos dentro de métodos	7, en 3 métodos
Líneas de control de acceso que no se ejecutan nunca	219
Líneas triplicadas en los tres inventarios	192
Excepciones envueltas	32 sitios en 14 clases
Vistas acopladas a clases concretas	7 de 22
Líneas de <code>PersistenciaService</code>	654

5. Pendiente antes de entregar

- ☐ Marcar cuáles tres puntos se encontraron sin IA. El enunciado lo exige y la rúbrica lo audita al azar. La evidencia es la lectura individual previa en `00-lectura-en-frío/`; los que salgan de ahí se registran en la bitácora de la Actividad 2.
- ☐ Elegir cuáles cinco de los siete van al PDF final.
- ☐ Confirmar que N-02 es el punto no intervenido, o cambiarlo por N-01 o N-03.
- ☐ Verificar que cada punto elegido tiene un patrón anclado en la Actividad 2. Un patrón sin P-xx detrás cuesta 0.3 sobre la nota final.